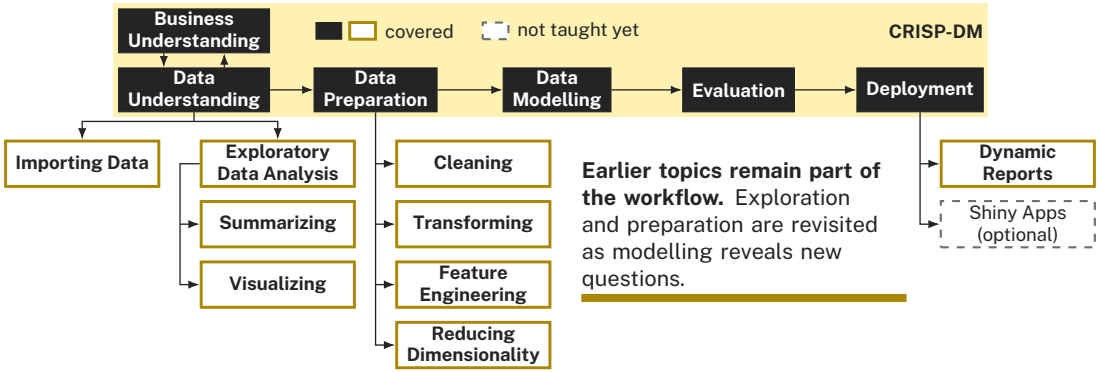
DALHOUSIE UNIVERSITY

# Predictive Analytics (cont.)

Evaluation metrics, and the traps inside them

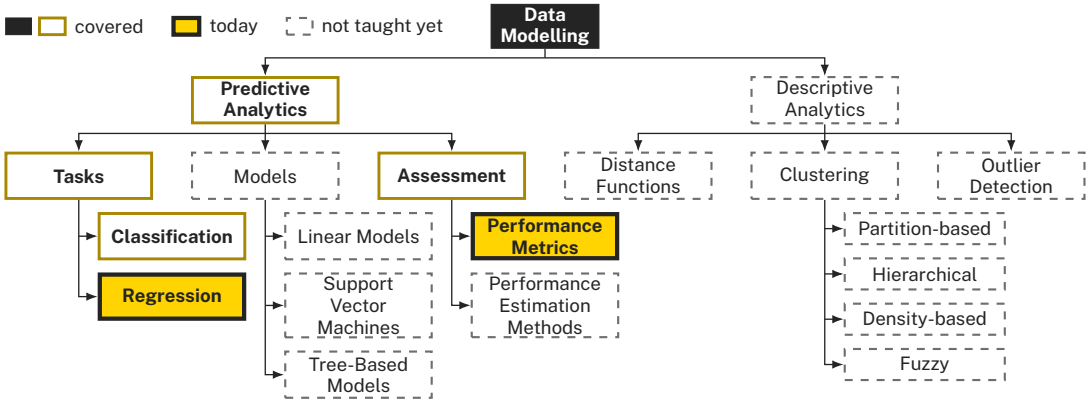
**Prof. Gabriel Spadon, PhD**
Faculty of Computer Science
spadon@dal.ca

# The course map so far




**Earlier topics remain part of the workflow.** Exploration and preparation are revisited as modelling reveals new questions.

# Today's performance metrics




Today covers metrics for a numeric target, from absolute and relative errors to $R^2$ and percentage errors.

## Absolute error metrics

DALHOUSIE UNIVERSITY

An absolute error metric averages the size of each error, in the target's units or their square.

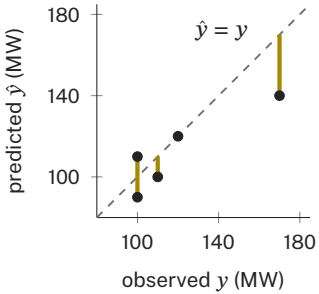
# One test set, every metric

Five constructed test days. $y_i$ is observed peak load in MW; $\hat{y}_i$ is the prediction. The regression metrics use this table; later variations are labelled.

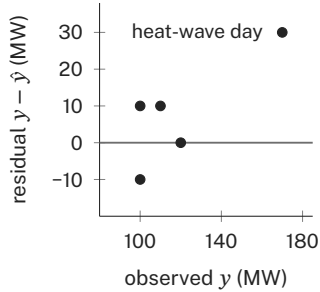
| $i$ | $y_i$ | $\hat{y}_i$ | $e_i = y_i - \hat{y}_i$ | $\lvert e_i \rvert$ | $e_i^2$ | $\lvert \bar{y} - y_i \rvert$ | $(\bar{y} - y_i)^2$ |
|---|---|---|---|---|---|---|---|
| 1 | 100 | 110 | −10 | 10 | 100 | 20 | 400 |
| 2 | 110 | 100 | +10 | 10 | 100 | 10 | 100 |
| 3 | 100 | 90 | +10 | 10 | 100 | 20 | 400 |
| 4 | 120 | 120 | 0 | 0 | 0 | 0 | 0 |
| 5 | 170 | 140 | +30 | 30 | 900 | 50 | 2500 |
| Σ | 600 | 560 | +40 | **60** | **1200** | **100** | **3400** |

$N_{test} = 5$; $\bar{y} = 600/5 = 120$ MW. Rows 1–3 show MAPE and sMAPE calculations; row 5 is the day the model badly under-predicts.

# Look before you average




Predicted against observed, with residuals in gold.




Residuals against observed; one point dominates.

Four errors of at most 10 MW and one of 30 MW. Every metric that follows is a different answer to *how much should that fifth point count?*

# Mean Squared Error (MSE)

$$MSE = \frac{1}{N_{test}} \sum_{i=1}^{N_{test}} (y_i - \hat{y}_i)^2$$

where $\hat{y}_i$ is the prediction of the model under evaluation for the case $i$ and $y_i$ the respective true target variable value.

## ON THE RUNNING EXAMPLE

$$MSE = \frac{100 + 100 + 100 + 0 + 900}{5}$$

$$= \frac{1200}{5} = 240 \text{ MW}^2$$

The single 30 MW error contributes 900 of the 1200, **75%** of the total from **20%** of the cases. Squaring is what does that.

# MSE for Fitting, RMSE for Reporting

$$MSE = \frac{1}{N_{test}} \sum_{i=1}^{N_{test}} (y_i - \hat{y}_i)^2 \qquad RMSE = \sqrt{MSE}$$

- Differentiable.
  - Makes it easier to use as the evaluation criterion to optimize a model.
- Sensitive to outliers, as it punishes larger errors more.
- Not in the same units as the $Y$ variable.
  - Can use Root Mean Squared Error (RMSE) instead.

## ON THE RUNNING EXAMPLE

$$RMSE = \sqrt{240} = 15.49 \text{ MW}$$

240 is in $MW^2$ and is harder to interpret operationally. RMSE is 15.49 MW, an error measure in the units of the quantity predicted.

# Mean Absolute Error (MAE)

$$MAE = \frac{1}{N_{test}} \sum_{i=1}^{N_{test}} |y_i - \hat{y}_i|$$

The MAE measures the average absolute deviation between the predictions and the true values, in the same units as $Y$ itself.

- Not differentiable at $y_i = \hat{y}_i$.
  - Harder to use as the evaluation criterion to optimize a model.
- More robust to outliers.
- Same units as variable $Y$.

## ON THE RUNNING EXAMPLE

$$MAE = \frac{10 + 10 + 10 + 0 + 30}{5}$$

$$= \frac{60}{5} = 12 \text{ MW}$$

Sensitivity to outliers helps when a large error costs more than proportionally, and misleads when the large errors are recording mistakes.

# What RMSE and MAE each answer

DALHOUSIE UNIVERSITY

## MAE $= 12$ MW

- *On a typical day, how far off am I?*
- Every megawatt of error counts once, wherever it occurs.
- The conditional **median** minimizes MAE.

## RMSE $= 15.49$ MW

- *How exposed am I to a bad day?*
- A 30 MW error costs nine times a 10 MW error.
- The conditional **mean** minimizes RMSE.

Repair the worst day only. Change $\hat{y}_5$ from 140 to 160 and leave every other row unchanged.

| | before | after | change |
|---|---|---|---|
| MAE | 12.00 MW | 8.00 MW | $-33\%$ |
| RMSE | 15.49 MW | 8.94 MW | $-42\%$ |

Always $MAE \le RMSE \le \sqrt{N_{test}} \cdot MAE$; equality on the left only when every error is the same size. **Report both**; the gap between them shows how much the largest errors dominate.

# MSE, RMSE and MAE in code

## R

```r
y <- c(100, 110, 100, 120, 170)
p <- c(110, 100,  90, 120, 140)
mean(abs(y - p))          # MAE   12
mean((y - p)^2)           # MSE   240
sqrt(mean((y - p)^2))     # RMSE  15.49193
library(Metrics)
mae(y, p); mse(y, p); rmse(y, p)   # same three
caret::postResample(pred = p, obs = y)
#       RMSE   Rsquared        MAE
# 15.4919334  0.7949126 12.0000000
```

## Python

```python
import numpy as np
from sklearn.metrics import (mean_absolute_error,
    mean_squared_error, root_mean_squared_error)
y = np.array([100, 110, 100, 120, 170])
p = np.array([110, 100,  90, 120, 140])
mean_absolute_error(y, p)         # 12.0
mean_squared_error(y, p)          # 240.0
root_mean_squared_error(y, p)     # 15.491933
np.sqrt(np.mean((y - p) ** 2))    # 15.491933
```

# Absolute-Error Defaults That Bite

- `squared=False` was the old route to RMSE, deprecated in scikit-learn 1.4 and *removed* in 1.6; it now raises `TypeError`. Use `root_mean_squared_error`.
- No metric here divides by $n-1$, but normalize by a variance and it matters. `var()` uses $n-1$ and `np.var()` uses $n$, so `MSE/var(y)` is $0.282$ in R and $0.353$ in NumPy, and only the second is the NMSE of the next section.
- `postResample` labels the *squared correlation* `Rsquared`, which gives $0.795$ here. The residual $R^2$ defined later in this class gives $0.647$.

## Relative error metrics

DALHOUSIE UNIVERSITY

A relative metric divides the model's error by the error of a named baseline.

# NMSE and NMAE

Divide model error by baseline error. Here the baseline uses the *test-target mean*, $\bar{y} = 120$ MW. A deployable mean baseline uses training data; compare the choices next.

$$NMSE = \frac{\sum_{i=1}^{N_{test}} (y_i - \hat{y}_i)^2}{\sum_{i=1}^{N_{test}} (\bar{y} - y_i)^2}$$

$$NMAE = \frac{\sum_{i=1}^{N_{test}} |y_i - \hat{y}_i|}{\sum_{i=1}^{N_{test}} |\bar{y} - y_i|}$$

Both are unitless. Name the baseline; a zero denominator makes either undefined.

## ON THE RUNNING EXAMPLE

$$NMSE = \frac{1200}{3400} = 0.353$$

$$NMAE = \frac{60}{100} = 0.600$$

$< 1$ beats the baseline, $= 1$ ties it, $> 1$ loses to it. Here squared error is **35%** and absolute error **60%** of baseline error.

# Name your baseline

Two conventions exist for $\bar{y}$, the training-set average or the test-set average, scoring the same predictions on the same test set two different ways.

| baseline $\bar{y}$ | denominator | NMSE |
|---|---|---|
| test mean, 120 | 3400 | 0.353 |
| training mean, 130 | 3900 | 0.308 |

Neither is wrong. Reporting the number without saying which one you used is.

With the test mean as the baseline, $1 - NMSE$ is called $R^2$.

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = 1 - \frac{1200}{3400} = 0.647,$$

with $SS_{res} = \sum(y_i - \hat{y}_i)^2$ and $SS_{tot} = \sum(y_i - \bar{y})^2$.

$R^2$ can be negative. Predicting a constant 100 MW gives $SS_{res} = 5400 > 3400$, so

$$R^2 = 1 - \frac{5400}{3400} = -0.588.$$

On a test set nothing keeps $R^2$ above 0. A negative value means *worse than the mean*.

# $R^2$ is not always $r^2$

DALHOUSIE UNIVERSITY

## RESIDUAL DEFINITION

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2} = 0.647$$

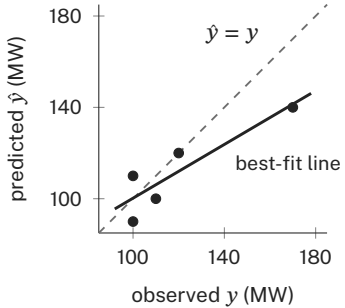
Punishes bias and mis-scaling. Used by `r2_score` in scikit-learn and by `caret::R2(formula = "traditional")`.

## CORRELATION DEFINITION

$$r^2 = \left(\frac{\operatorname{cov}(y, \hat{y})}{s_y\, s_{\hat{y}}}\right)^2 = 0.8916^2 = 0.795$$

Blind to bias and to scale. Reported by `caret::postResample` by default.

# Seeing the gap between $R^2$ and $r^2$




The points track a line of slope $0.59$, not the line $\hat{y} = y$. Correlation sees only the tracking. $r^2$ ignores bias and scale, so here it exceeds $R^2$ by $0.15$.

# Mean Absolute Percentage Error

$$MAPE = \frac{1}{N_{test}} \sum_{i=1}^{N_{test}} \frac{|y_i - \hat{y}_i|}{y_i}$$

- Undefined at a zero target; can be extremely high near zero.
- Not differentiable at $y_i = \hat{y}_i$; needs a nonsmooth optimization method.
- Here targets are positive, on a scale with a meaningful zero.
- Asymmetric, exchanging $y_i$ and $\hat{y}_i$ yields a different MAPE.
- Unbounded for over-predictions; at most 100% for nonnegative under-predictions.

$y_i = 100, \hat{y}_i = 110$: MAPE $= 0.100$
$y_i = 110, \hat{y}_i = 100$: MAPE $= 0.091$

## ON THE RUNNING EXAMPLE

$$\frac{0.100 + 0.091 + 0.100 + 0 + 0.176}{5}$$
$$= 0.0935 = 9.35\%$$

Rows 1 and 2 of the running example illustrate the key edge case.

# Reading MAPE's asymmetry

## THE DENOMINATOR IS THE TRUTH

With $y$ *held fixed*, MAPE treats over- and under-prediction of the same size identically, $y = 100$ scores $0.1$ for $\hat{y} = 110$ and $0.1$ for $\hat{y} = 90$.

When roles are swapped, the denominator changes. The pair $\{100, 110\}$ scores $10/100 = 0.1$ one way and $10/110 = 0.09$ the other. For the same absolute error, MAPE is larger when the true value is smaller.

## THE REAL CONSEQUENCE

Since $\hat{y} \geq 0$, under-prediction can never exceed $100\%$ while over-prediction is unbounded. MAPE weights absolute errors by $1/y$, so small targets count more. This can favor lower forecasts than MAE or MSE.

# MAPE at a zero

## THE ZERO

Add one quiet hour with $y_6 = 0$ and $\hat{y}_6 = 5$ to the running example. The sixth term is $5/0$, so the mathematical metric is undefined; direct floating-point division returns $\infty$. The table shows what each library returns.

| | |
|---|---|
| `mean(abs((y-p)/y))` | `Inf` |
| `Metrics::mape` (R) | `Inf` |
| `sklearn mean_absolute_percentage_error` | **3.753e+15**, no error, no warning |

scikit-learn replaces $y_i$ by $\max(|y_i|, \varepsilon)$ with $\varepsilon \approx 2.2 \times 10^{-16}$. The result is finite, enormous, and easy to mistake for a real score.

# Symmetric MAPE (sMAPE)

$$sMAPE = \frac{1}{N_{test}} \sum_{i=1}^{N_{test}} \frac{|y_i - \hat{y}_i|}{|y_i| + |\hat{y}_i|}$$

- Undefined if $y_i = \hat{y}_i = 0$; sensitive to small changes near zero, but bounded by 1.
- Still assumes the unit of measurement has a "meaningful zero" value.
- Fixes one type of asymmetry, exchanging $y_i$ and $\hat{y}_i$ yields the same sMAPE.
- Asymmetric in a different way, fixing the true value and moving the same absolute deviation in different directions yields a different sMAPE.

$y_i = 100, \hat{y}_i = 110$: sMAPE $= 4.76\%$
$y_i = 100, \hat{y}_i = 90$: sMAPE $= 5.26\%$

## ON THE RUNNING EXAMPLE

$$\frac{0.0476 + 0.0476 + 0.0526 + 0 + 0.0968}{5}$$

$$= 0.0489 = 4.89\%$$

Rows 1 and 2 both score $10/210 = 4.76\%$, so swapping truth and prediction leaves sMAPE unchanged.

# Relative metrics in code

## R

```r
sum((y - p)^2)  / sum((y - mean(y))^2)
# NMSE 0.3529412
sum(abs(y - p)) / sum(abs(y - mean(y)))
# NMAE 0.6
mape(y, p)    # 0.09347594
smape(y, p)   # 0.09785755, doubled convention
```

## Python

```python
from sklearn.metrics import (
    mean_absolute_percentage_error)
1 - r2_score(y, p)   # NMSE 0.3529412
mean_absolute_percentage_error(y, p)
# 0.09347594, a fraction
# no NMAE and no sMAPE in scikit-learn
np.sum(np.abs(y - p)) / np.sum(
    np.abs(y - y.mean()))   # NMAE 0.6
np.mean(np.abs(y - p) / (np.abs(y) + np.abs(p)))
# sMAPE 0.04892877
```

# Relative-Metric Defaults That Bite

DALHOUSIE UNIVERSITY

- sMAPE may divide by the sum (0–100%) or by half the sum (0–200%); `Metrics::smape` uses the doubled form.
- The explicit NMSE and NMAE use the test mean; substitute the training mean when that is the baseline you intend.
- R propagates `NA`; sklearn raises on `NaN`; NumPy returns `nan` unless you request a missing-aware mean.

# Nine numbers from one test set

| Metric | Value | Units | What it answers |
|---|---|---|---|
| MAE | 12.00 | MW | Typical size of an error. |
| MSE | 240.00 | $MW^2$ | Average squared error; fitting criterion. |
| RMSE | 15.49 | MW | Typical error, with large errors weighted up. |
| NMSE | 0.353 | none | Squared error as a fraction of the baseline's. |
| NMAE | 0.600 | none | Absolute error as a fraction of the baseline's. |
| $R^2 = 1 - SS_{res}/SS_{tot}$ | 0.647 | none | Test-mean improvement; $= 1 - NMSE$. |
| $r^2$ (squared correlation) | 0.795 | none | How well predictions *track*, ignoring bias. |
| MAPE | 9.35% | % | Error relative to each true value. |
| sMAPE | 4.89% | % | Error relative to truth and prediction together. |

These nine numbers summarize one test set. Choose the metric before seeing results; report its baseline and units.

# What to take away

- Each metric has its own pros and cons.
- MSE is differentiable, which makes it easier to use as a model optimization criterion.
- RMSE and MAE are expressed in the same units as the target variable.
- Relative metrics ease comparison across data sets with different scales.
- Name the baseline of a normalized metric, say which $R^2$ your software reported, and check percentage-metric denominators for zeros.